

Практическая работа №4

Выполнила: Ушакова Е.А. гр. ЗП-31

Все исходные файлы лабораторных работ лежат в папке репозитория по ссылке
<https://github.com/irbi42/uni/tree/main/vp/labs>

1) Динамически создаем виджет запуска скриптов при открытии приложения. В его компоновку входят кнопки выполнить, остановить и поле PlainTextEdit, настроенное только на вывод информации

```
ui->plainTextEdit->document()->setModified(false);
pyRunnerWidget = new QWidget();
pyRunnerWidget->setMinimumWidth(400);
runButton = new QPushButton("Выполнить", pyRunnerWidget);
stopButton = new QPushButton("Остановить", pyRunnerWidget);
QHBoxLayout *pyRunnerLayout = new QHBoxLayout();
QVBoxLayout *pyRunnerButtonLayout = new QVBoxLayout();
runnerOutput = new QPlainTextEdit();
runnerOutput->setReadOnly(true);
pyRunnerButtonLayout->addWidget(runButton);
pyRunnerButtonLayout->addWidget(stopButton);
pyRunnerLayout->addLayout(pyRunnerButtonLayout);
pyRunnerLayout->addWidget(runnerOutput);

pyRunnerWidget->setLayout(pyRunnerLayout);

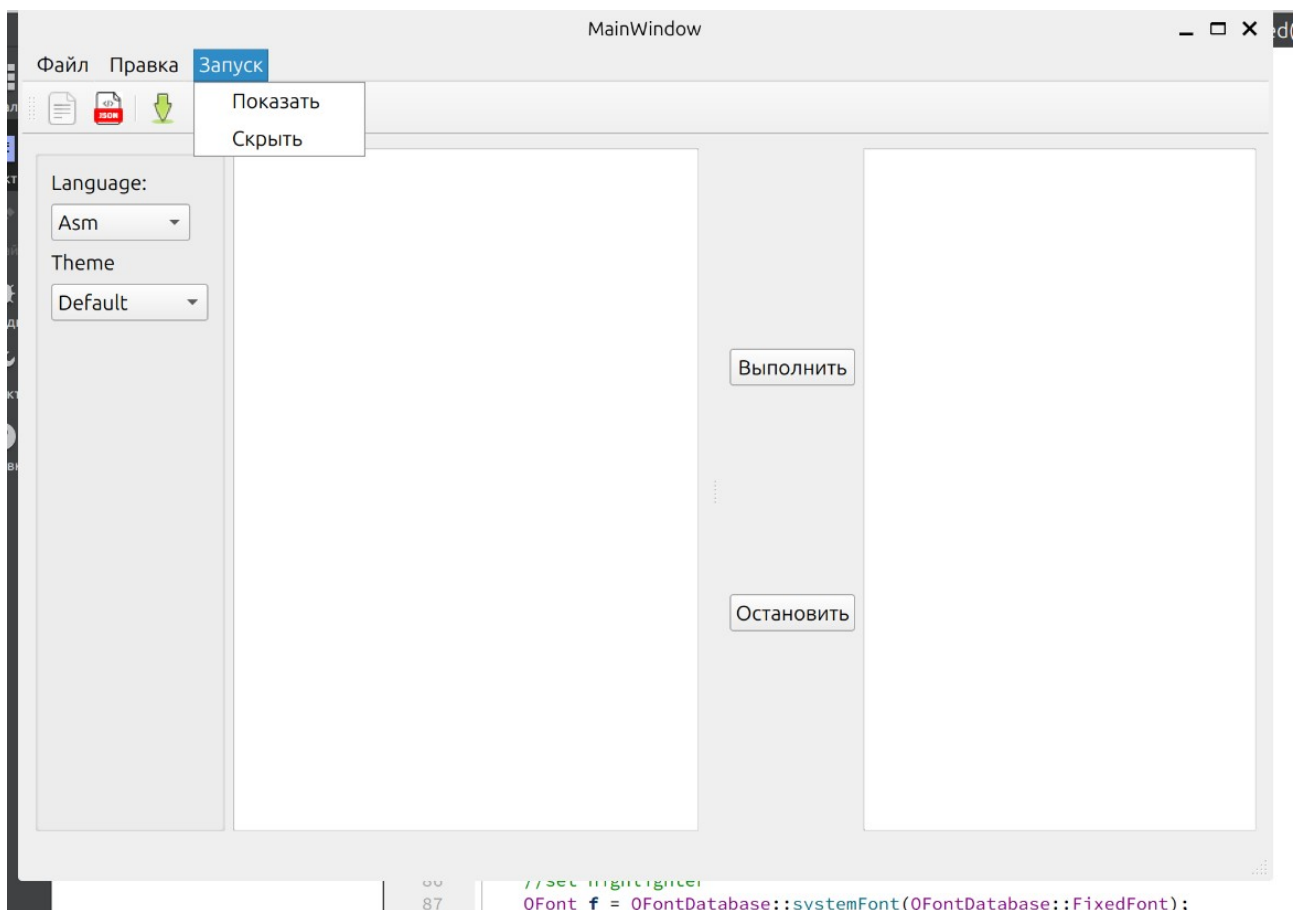
QSplitter *mainSplitter = new QSplitter();
mainSplitter->addWidget(ui->centralWidget);
ui->centralWidget->setMinimumWidth(500);
mainSplitter->addWidget(pyRunnerWidget);

setCentralWidget(mainSplitter);

pyRunnerWidget->hide();
```

Виджет скрыт по умолчанию и показывается через выпадающее меню Запуск.

```
connect(ui->showRunner, &QAction::triggered, this, &MainWindow::onShowRunner);
connect(ui->hideRunner, &QAction::triggered, this, &MainWindow::onHideRunner);
connect(runButton, &QPushButton::clicked, this, &MainWindow::onRunScript);
connect(stopButton, &QPushButton::clicked, this, &MainWindow::onStopScript);
```



2) Запуск скрипта осуществляется через слот onRunScript()

```

191 void MainWindow::onRunScript() {
192     if (ui->langComboBox->currentText() != "Python") {
193         QMessageBox::information(this, "Ошибка", "Функционал не доступен для выбранного языка");
194         return;
195     }
196
197     if (process.state() == QProcess::Running) {
198         QMessageBox::information(this, "Предупреждение", "Процесс уже запущен. Сначала остановите текущий.");
199         return;
200     }
201
202     QString fileName;
203     if (currentFile.isEmpty() || ui->plainTextEdit->document()->isModified()) {
204         fileName = "temp.py";
205     } else {
206         fileName = currentFile;
207     }
208     QFile file(fileName);
209     if (!file.open(QIODevice::WriteOnly | QIODevice::Text)) {
210         ui->statusbar->showMessage("Ошибка: не удалось открыть файл для записи.", 3000);
211         return;
212     }
213     QString text = ui->plainTextEdit->toPlainText();
214     QTextStream out(&file);
215     out << text;
216
217     file.close();
218     runnerOutput->clear();
219
220     process.start("python3", QStringList() << fileName);
221
222     if (!process.waitForStarted()) {
223         QMessageBox::critical(this, "Ошибка", "Python не установлен или не найден в PATH");
224         return;
225     }
226
227     QTimer::singleShot(3000, this, [this, fileName]() {
228         if (process.state() == QProcess::Running) {
229             process.kill();
230             process.waitForFinished(1000);
231             runnerOutput->appendPlainText("Превышено время выполнения (3 сек). Процесс остановлен.\n");
232         }
233         if (fileName == "temp.py" && QFile::exists(fileName)) {
234             QFile::remove(fileName);
235         }
236     });
237 }
238

```

функция проверяет соответствие выбранного языка, наличие других запущенных процессов. Затем содержимое файла записывается в файл и запускается Qprocess. Если скрипт не завершился в течение 3 сек., процесс останавливается. Результат выводится при получении сигнала от процесса о завершении или наличии вывода в стандартном выводе.

```

118 connect(&process, &QProcess::readyReadStandardOutput, this, [this]() {
119     runnerOutput->appendPlainText(process.readAllStandardOutput());
120 });
121
122 connect(&process, &QProcess::readyReadStandardError, this, [this]() {
123     runnerOutput->appendPlainText("[ERROR] " + process.readAllStandardError());
124 });
125
126 connect(&process, QOverload<int, QProcess::ExitStatus>::of(&QProcess::finished),
127     this, [this](int exitCode, QProcess::ExitStatus status) {
128         if (status == QProcess::NormalExit && exitCode == 0) {
129             runnerOutput->appendPlainText("\nСкрипт завершен");
130         } else if (status == QProcess::NormalExit && exitCode != 0) {
131             runnerOutput->appendPlainText(QString("\nКод возврата: %1").arg(exitCode));
132         }
133         if (QFile::exists("temp.py")) {
134             QFile::remove("temp.py");
135         }
136     });
137 }

```

Кнопка остановки вызывает функцию, останавливающую процесс.

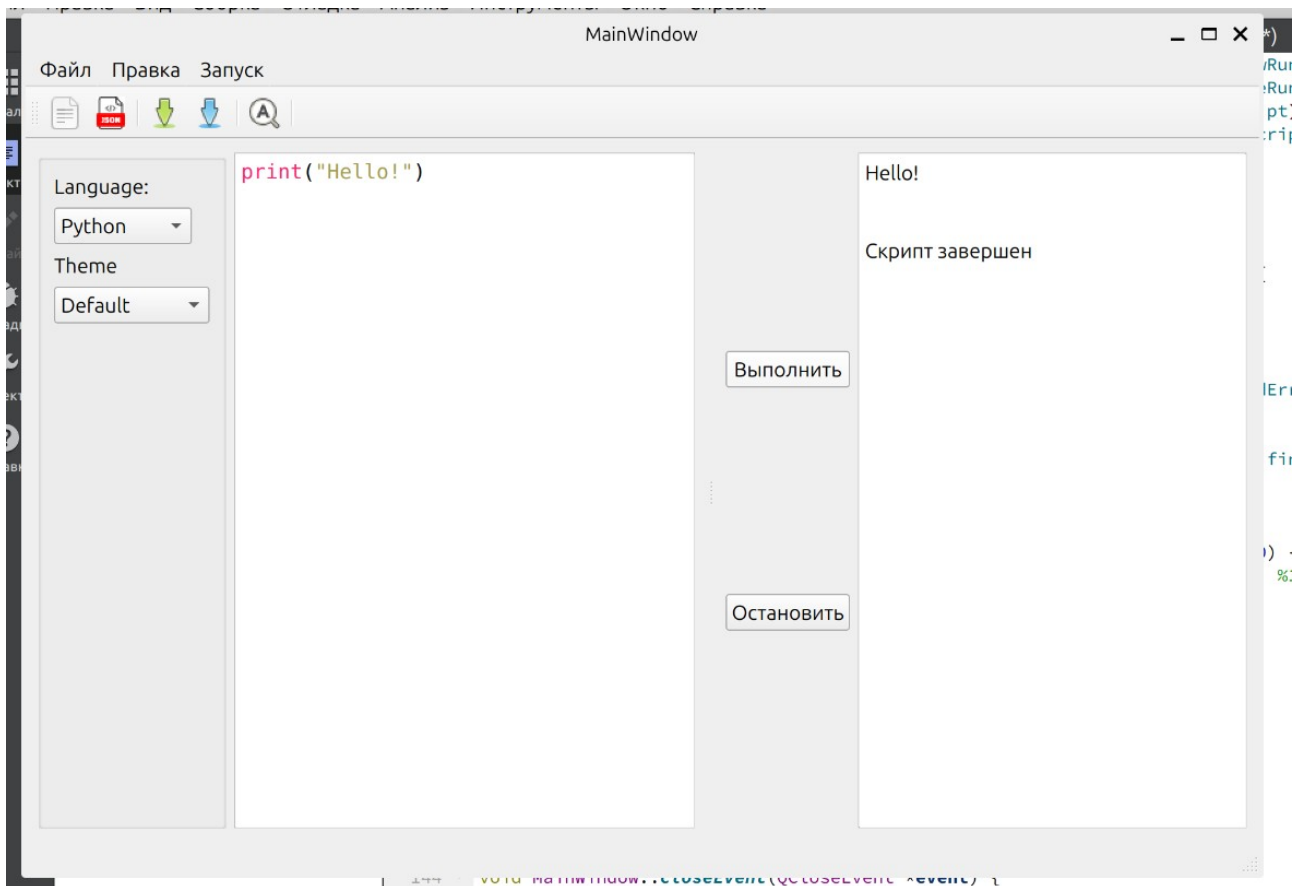
```

    }
}

void MainWindow::onStopScript() {
    if(process.state() == QProcess::Running) {
        process.kill();
        runnerOutput->setPlainText(runnerOutput->toPlainText() + "Процесс остановлен.\n");
        ui->statusbar->showMessage("Процесс остановлен.", 3000);
    }
}

```

Результат работы:



Ответы на контрольные вопросы

Что такое QProcess и зачем он используется в Qt?

QProcess — класс для запуска внешних программ из Qt-приложения. Используется для выполнения команд, скриптов и получения их вывода.

Что означают параметры при вызове функции process.start()?

process.start(программа, аргументы). Первый параметр — путь к исполняемому файлу. Второй (опционально) — список аргументов командной строки.

Как проверить, успешно ли запущен процесс?

Вызвать process.waitForStarted() (вернёт true/false) или подключить к сигналу started(). Также проверить состояние process.state() == QProcess::Running.

Как получить вывод программы после её выполнения?

Через `process.readAllStandardOutput()` (`stdout`) и `process.readAllStandardError()` (`stderr`). Обычно после `waitForFinished()` или по сигналам `readyReadStandardOutput()`.

Почему важно использовать временные файлы при запуске пользовательского кода?

Для безопасности и изоляции: пользовательский код не должен изменять файлы приложения, временные файлы автоматически удаляются и не засоряют систему.

Как работает метод `waitForFinished()` и зачем он нужен?

Блокирует выполнение текущего потока до завершения процесса. Нужен для синхронного ожидания результата. Возвращает `true`, если процесс завершился за указанное время.

Как очистить временный файл после выполнения скрипта и как можно использовать при этом `QTemporaryFile`?

`QTemporaryFile` автоматически удаляет файл при разрушении объекта (выход из области видимости). Вручную можно вызвать `remove()`. Использование: `QTemporaryFile temp; temp.open(); temp.write(скрипт); temp.close(); process.start(temp.fileName());`

Как обрабатывать ситуации, когда Python-скрипт завис или выполняется слишком долго?

Использовать `waitForFinished(таймаут)`. Если возвращает `false` — процесс не завершился вовремя, вызвать `process.kill()` или `process.terminate()`, затем показать сообщение об ошибке.